Library-Independent Adjudication of Three Rigid-Body Dynamics Engines: Agreement, Divergence in Reach, and a Shared Failure at Unstable Equilibria

Noah Parsons

Independent Researcher, Newcastle, Wyoming

ORCID 0009-0000-7224-6040

September 5, 2026

Abstract

Differential testing detects the failures that independent implementations make independently. It is structurally blind to the failures they share, and this paper demonstrates that such failures exist in widely used scientific software and are not reachable by adding implementations. Three rigid-body dynamics engines — a symbolic compiler, a general-purpose computer-algebra mechanics package, and a recursive articulated-body library — were compared against a closed-form reference implemented in NumPy alone that shares no library with any of them, a stronger independence criterion than independent authorship. On every adjudicated case the engines agree, and on one configuration all three fail identically — as does the reference, which is what makes the failure invisible to the method.

A 55-case adversarial suite spanning six axes was frozen before measurement, and the comparison was extended to three structurally distinct mechanisms: an N-link chain exercised from its regular regime into chaos, a slider–crank whose closed loop and prismatic joint produce dead-centre configurations where the effective inertia varies by four orders of magnitude over one revolution, and a cart–pole whose mass-matrix determinant collapses by six orders of magnitude twice per swing. Integration was pinned to a single scheme with identical tolerances for all engines, which removes the integrator as a variable and, by construction, forecloses any ranking of the engines on integration accuracy.

The engines agree with the reference and with one another to within 3×10^{-14} on the equations of motion themselves, across both dynamical regimes and across constrained and unconstrained pathways. Energy drift is likewise indistinguishable, but that is a consistency check rather than independent evidence: all four share the pinned integrator, so near-identical drift is close to guaranteed by construction. The weight rests on the acceleration residuals, which do not. Every engine that returned an answer returned a correct one, and no engine was observed to return a wrong answer while reporting success. The engines were initially found to differ in reach — returning equations for chains of 5, 12 and 30 links — but that separation proved to be an artefact of how each engine was driven rather than a property of the engines: given the same systems and the same budget, all three answer chains of 80 links (§6).

One failure was observed, and it is shared. At an unstable equilibrium, all three engines and the independent reference report success while returning approximately 20 radians of motion for a configuration whose exact solution does not move; none warns that the trajectory is dominated by round-off amplified by the instability. Silent failure in these tools is therefore a property of finite-precision dynamics rather than a property that distinguishes implementations — a result that constrains what differential testing between implementations can be expected to reveal about correctness in this domain.

Contents

1 Introduction

1.1 The problem

Scientific results increasingly rest on software that derives the equations governing a physical system and integrates them forward. A researcher modelling a joint, a mechanism, or a robot selects an engine, states the system, and takes the resulting trajectory as an account of what the physical system would do. The correctness of that account is rarely tested directly. It is inferred from the engine's reputation, from its test suite, and from the absence of an error message.

Where that inference is tested, it is usually tested by comparison: run two implementations and treat disagreement as evidence of a defect. The method is powerful and its limit is well known in principle — it can only find failures the implementations do not share. What that limit means in practice is less often examined, because it requires finding a shared failure and showing that no amount of implementation diversity would have exposed it. This paper does that, and the answer is not reassuring: the shared failure reported here originates in floating-point arithmetic rather than in anyone's code, so every implementation exhibits it, including a reference written from first principles specifically to be independent.

1.2 The failure mode that motivates the study

An engine that raises an exception is not dangerous; the researcher knows to stop. The dangerous case is the engine that returns a plausible trajectory that is wrong, with no indication that anything has gone amiss. Call this a *silent failure*. A silent failure is by construction the case nobody thought to check for, which is why it survives in test suites written by the same people who wrote the implementation.

1.3 Two claims, often conflated

Discussion of this failure mode tends to run together two distinct propositions:

(A) These engines can report success while returning wrong physics.

(B) Engines *differ* in whether they do so, such that comparing implementations reveals incorrectness.

Claim (B) is the premise of differential testing: if two independent implementations disagree, at least one is wrong, and the disagreement is the signal. This paper set out to test (B) and did not find it.

1.4 What was done

Three independently developed engines were compared against a closed-form reference implemented in NumPy alone, sharing no library with any of them. A 55-case adversarial matrix was frozen before measurement under a governance rule forbidding removal of cases after their results were seen. Three mechanism families were examined, chosen so that each degenerates by a different mechanism. Integration was pinned to one scheme with identical tolerances for every engine. The author's conflict of interest is disclosed in §3.7, and again in the Disclosure statement following the conclusion, and is mitigated structurally rather than by declaration.

1.5 What was found

Agreement to the limit of double-precision arithmetic on every adjudicated case in every family. Divergence between the engines in *reach* — the largest system each will answer — but not in correctness. One failure, at an unstable equilibrium, shared by all three engines and by the independent reference alike.

1.6 Why a negative result here is informative

The measurement was designed to detect disagreement, and the evidence that it can is its record against itself. It detected *four* faults in its own construction — three convention mismatches and one mistaken adjudication criterion — before any of them reached the reported results (§3.6, §9). Each was caught by consulting an independent artefact rather than by reasoning further, and each would otherwise have produced a fabricated finding against widely used software. An instrument that catches four faults in itself is not one that would miss a correctness disagreement of the size differential testing exists to expose.

A second argument was originally made here and is withdrawn. The instrument appeared to resolve a large capability difference between the engines, and that was offered as further evidence of its sensitivity. The difference was an artefact of how each engine was driven (§6). It is recorded here because a study of measurements that mislead should not quietly discard its own: the reach comparison was confounded, the confound survived the freeze, and it was found only when each engine was driven along more than one path. The correctness measurements are unaffected, because they adjudicate each engine against an independent reference at fixed size rather than against each other.

1.7 Contributions

1. A differential comparison of three independently developed rigid-body dynamics engines under adjudication by a reference sharing no library with any of them, across three mechanism families and both dynamical regimes.
2. A negative result: no correctness disagreement was found on any axis measured; the engines separate by reach, not by honesty.
3. A shared blind spot at unstable equilibria, present in all three engines *and* in the independent reference, showing that the failure mode is a property of finite-precision dynamics rather than of any implementation.
4. A methodological account — frozen case matrix, library-independent referee, integrator pinning, maintainer disclosure — reusable for cross-implementation comparison in other computational domains.
5. A transferable diagnostic for convention mismatch in cross-implementation comparison, derived from three independent instances encountered in this study.

2 Related work

2.1 Differential testing and the independence assumption

Differential testing — running two or more implementations of the same specification on the same input and treating any difference in output as evidence of a defect — was named and systematised by McKeeman [9], and has since become a standard instrument wherever a specification admits multiple implementations. Its appeal is that it dissolves the test oracle problem: no statement of correct behaviour is needed, only two implementations and a disagreement.

Its premise is that independent implementations fail independently. That premise has been tested directly, and it failed. Knight and Leveson [8] prepared 27 versions of one program independently from a common specification at two universities and subjected them to one million tests. The versions were individually very reliable, but the number of tests on which more than one version failed was substantially greater than independence predicts. Their result is the standard caution against N-version programming, and it is the closest antecedent of the present paper's central finding.

The mechanism differs, and the difference is the contribution. Knight and Leveson attributed correlated failure to shared human misconception: independent programmers reading the same specification make the same mistakes, because some parts of a problem are harder to think about than others. The correlated failure reported in §7 has no human cause at all. Every implementation examined here — including a reference written from first principles specifically to be independent — fails identically at an unstable equilibrium, because the failure originates in floating-point arithmetic rather than in anyone's reasoning. Shared misconceptions can in principle be diluted by adding more independent authors. Shared arithmetic cannot.

2.2 The oracle problem and scientific software

Barr et al. [2] survey the test oracle problem and classify the partial remedies available when no exact oracle exists: derived oracles, metamorphic relations, and *pseudo-oracles* — independently produced implementations used as a standard of comparison. The reference used here is a pseudo-oracle in that sense, distinguished by an explicit and checkable independence criterion: it shares no library with any engine under test, which is a stronger condition than independent authorship and a much stronger one than independent implementation within a common ecosystem.

Kanewala and Bieman [7] review testing practice in scientific software specifically and identify the oracle problem as one of its two defining difficulties: for software whose purpose is to compute a result nobody knows in advance, the expected output is frequently unavailable by construction. This is precisely the situation of a dynamics engine asked for the trajectory of a mechanism that has never been built.

The most direct empirical precedent is Hatton and Roberts [5], who ran nine independently developed seismic data processing packages on identical inputs and found agreement to roughly one significant figure — a disagreement so large that it called into question published results in the field. Their study and this one share a design and reach opposite conclusions, which requires explanation rather than assertion. Three differences are plausible. Their packages implemented a long processing pipeline whose specification left considerable latitude, whereas the equations of motion of a rigid mechanism are mathematically determined once the Lagrangian is fixed. Their comparison was engine-against-engine, with no external standard, so disagreement could be localised but not attributed. And three decades separate the two studies. The present result should therefore be read narrowly: rigid-body dynamics is a domain where the specification is unusually tight, and the finding of agreement does not transfer to computational domains where it is not.

2.3 Code verification and the V&V literature

Computational science has a mature body of work on exactly the question asked here, with a settled vocabulary, and this study is best located inside it rather than beside it. Oberkampf and Trucano [10]

and the subsequent monograph of Oberkampf and Roy [11] draw the distinction that organises the field: *code verification* asks whether a program correctly solves the equations it claims to solve, *solution verification* asks how accurate one particular computed solution is, and *validation* asks whether those equations describe physical reality. Roache [12] states the separation in its sharpest form — verification is mathematics, validation is physics — and ASME V&V 10 [1] codifies the resulting process for computational solid mechanics.

In that taxonomy this paper is a code-verification study and nothing else. No measurement here is compared against an experiment, so no validation claim is made or implied; the reference is a mathematical standard, not a physical one. Three consequences follow, and stating them in the literature's own terms is more honest than restating them in ad hoc language.

First, the standard instrument of code verification is the method of manufactured solutions [13], in which an exact solution is chosen in advance and a source term is added to the governing equations to make it exact. Its purpose is to supply a known answer for equations that possess no useful closed-form solutions. The systems studied here are unusual in not needing that device: a rigid mechanism of modest size has a genuine closed-form acceleration field, so the reference of §3.2 plays the role of a manufactured solution without manufacturing anything, and without the residual doubt that attaches to whether a source-augmented problem still exercises the same code paths as the original. Where closed forms are unavailable — contact, friction, large flexible systems — manufactured solutions remain the appropriate tool and this design does not extend.

Second, code verification in the V&V sense conventionally rests on *order-of-accuracy* testing: refine the discretisation and confirm that the error falls at the rate the scheme promises, which detects a broad class of defects that a single-resolution comparison cannot. This study deliberately does not do that. Pinning one integrator at fixed tolerances across all engines (§3.4) is what makes the derivations comparable, and it is also what forecloses an order-of-accuracy claim. What is verified here is the right-hand side — the equations of motion each engine derives — and not the discretisation applied to it. A reader from this tradition should read the results as consistency verification of the derived equations at a fixed operating point, which is a narrower statement than code verification is sometimes taken to mean.

Third, the frozen case matrix and its governance rule (§3.3) are an informal instance of the pre-specified verification plan that ASME V&V 10 [1] requires: the cases and the criteria are fixed before measurement, and cases may be added but not withdrawn once scored. The motivation there is the same as here — a verification result is worth little if the set of tests it passed was chosen after seeing which ones it failed.

2.4 Verification and validation of multibody codes

The multibody dynamics community maintains its own verification infrastructure. The IFToMM Technical Committee for Multibody Dynamics hosts a library of computational benchmark problems [6] intended to standardise comparison of simulation software on both accuracy and efficiency. That effort is complementary to this one but differs in its notion of ground truth: benchmark solutions are established by consensus among submitted solvers, which is a reasonable practice and a different epistemic basis from a closed-form reference that shares no code with any participant.

Nearest in subject matter, González et al. [4] examine the behaviour of augmented-Lagrangian and Hamiltonian formulations for constrained multibody systems in the neighbourhood of singular configurations, identifying the sources of numerical difficulty there and comparing formulations for robustness. Their singular configurations are the analogue of the dead centres of §4.2. The distinction is that they compare *formulations* within a controlled implementation, isolating the effect of mathematical choice, whereas this study compares *implementations* of nominally the same formulation, isolating the effect of engineering. The two questions are separable and both worth asking; a formulation that is robust in principle can still be implemented incorrectly, and this paper measures the second.

2.5 Comparisons of dynamics engines

Erez, Tassa and Todorov [3] compare five simulation engines — among them Bullet, MuJoCo, ODE and PhysX — on quantitative measures chosen for robotics workloads, instantiating the same model in each and running side-by-side. Their principal conclusion is that each engine performs best on the class of system it was designed and optimised for. That finding is reproduced incidentally here, and by a different measurement: the recursive articulated-body library returns the largest residuals of the three engines on long serial chains and the smallest on the two-body tree (§5.6), an ordering that reverses with the shape of the problem rather than with the quality of the engine.

The difference in aim is what leaves room for the present study. Erez et al. measure speed, stability, and behaviour under contact — properties that matter for whether an engine is usable — and adjudicate against no external standard, because for the contact-rich systems they consider none exists in closed form. This paper asks a narrower question, on systems chosen precisely because closed forms do exist: not whether the engines are usable but whether their equations are right, judged against a derivation that shares nothing with any of them.

2.6 The gap

Prior work establishes that differential testing is powerful, that its independence premise is unreliable in the presence of shared human error, that scientific software has historically disagreed badly under N-version comparison, that dynamics engines differ measurably in performance, and that code verification has a mature methodology built largely around manufactured solutions and order-of-accuracy testing. What has not been reported, to the author's knowledge, is a comparison of symbolic and numerical dynamics engines adjudicated by a closed-form reference sharing no library with any of them, over a case matrix frozen before measurement, with the resulting disagreements characterised. That is the gap this paper addresses, and the result — agreement everywhere except in a failure the referee shares — is informative chiefly because the design would have detected the alternative.

3 Method

3.1 The engines under test

Three engines were selected on the criterion that each derives the equations of motion of a mechanical system by a different route, and that no two share a substantial body of code.

Engine	Route	Pathways exercised
MechanicsDSL 2.1.3 (`a8dc2b2`)	symbolic compiler; consumes a Lagrangian, direct solve for a fixed topology	Lagrangian, Hamiltonian, constrained
`sympy.physics.mechanics` 1.14.0	general-purpose computer algebra, driven as documented	Lagrangian, constrained
Drake 1.56.0	recursive articulated-body library for arbitrary topologies, contact, real-time control	model-building interface

Versions were pinned for the duration of the study. Pinning is not a formality: an engine that changes between the derivation and integration phases of a comparison invalidates the comparison, and the version under test must be recoverable by a reader attempting to reproduce the measurement.

Each engine was exercised through the pathways it exposes. Two support a Lagrangian formulation; one additionally supports a Hamiltonian formulation; one constructs models through a model-building interface rather than from a Lagrangian. This asymmetry is a property of the engines and is reported rather than normalised away, since normalising it would require driving at least one engine in a manner its documentation does not describe.

3.2 The referee

Adjudication is performed by a closed-form reference derived by hand and implemented in NumPy alone. This is the central design decision of the study, and its rationale is straightforward: two implementations that share a symbolic algebra layer, a linear solver, or a code-generation backend may agree because they inherit the same defect, and their agreement then carries no information about correctness.

The independence criterion, stated exactly. The reference shares no symbolic algebra system, no mechanics library, no code-generation backend and no derivation code with any engine under test. Every equation of motion it evaluates was derived by hand and transcribed. This is a stronger criterion than independent authorship, which permits two authors to import the same package, and much stronger than independent implementation within one ecosystem.

The referee is corroborated from outside the study. Adjudication by a single reference concentrates risk: if it is wrong, every engine agreeing with it is evidence of nothing. Its eleven self-tests do not dispose of this, since self-tests share the author's misconceptions — the objection Knight and Leveson [8] raise against independent implementation. The reference was therefore checked against a classical result predating the study: a uniform chain of length L hanging under gravity has small-oscillation frequencies $\omega_k = (z_k/2)\sqrt{g/L}$, where z_k is the k-th zero of J_0. Linearising the reference's own mass matrix and potential and taking N to the continuum reproduces it: relative errors on the first three modes fall from 5.5%, 4.3%, 1.6% at $N = 4$ to 0.19% on all three at $N = 128$, halving as N doubles. The reference reproduces a result it was never fitted to.

It is not, and cannot be, total independence, and overstating it would be the kind of claim this paper exists to scrutinise. The reference shares NumPy for array arithmetic with the evaluation paths of all three engines; it shares the pinned integrator (§3.4); and it shares IEEE-754 binary64 semantics and the host processor. What it does not share is any code that decides *what* the equations of motion are, which is the quantity under adjudication.

That distinction is load-bearing for §7, where a failure common to the engines *and* the reference is reported. A reader is entitled to ask whether such a failure is shared because the phenomenon is structural or merely because the substrate is. §7.3 answers that question by measurement rather than by assertion.

For the unconstrained families the reference solves the equations of motion directly for a fixed topology. For the constrained families both systems admit a constant diagonal mass matrix and holonomic constraints, so accelerations and multipliers follow from the saddle-point system

$$\begin{bmatrix} M & J^\top \\ J & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} F(q) \\ -\dot{J}\dot{q} \end{bmatrix}, \tag{1}$$

in which both J and $\dot{J}\dot{q}$ have closed forms for the rod and circle constraints involved. No symbolic algebra is required at any point in the reference, which is what permits the claim of library independence to be made literally rather than approximately.

The reference is deliberately left unstabilised: no Baumgarte term and no projection. A stabilised reference would be solving different equations from those the engines are asked for, and the resulting comparison would be between two different problems. The cost of that choice is a constraint drift of order 10^{-4} over three seconds, which does not affect the derivation comparison because that comparison evaluates accelerations at given states.

3.3 The case matrix and the freeze

The case matrix comprises 55 cases over 36 systems across six axes: degrees of freedom, closed-loop topology, constraint redundancy, near-singular inertia, mass ratio, and symbolic nesting depth. The

matrix was frozen — fixed in content and recorded with a version identifier — before the cross-engine measurements reported here were taken.

The freeze is governed by an explicit rule. Extension of the harness is permitted where it adds adjudication without revising existing verdicts, and any new axis must be declared before it is measured. Removing a case after observing its result is selection on the outcome and is forbidden. The rule exists because the author maintains one of the engines under test, and the temptation it guards against is therefore not hypothetical.

Of the 55 cases, 45 are adjudicated; the remainder exceed the wall-clock limit for at least one engine and are recorded as timeouts rather than as verdicts. A timeout is scored as a distinct category from a pass or a failure, because an absence of an answer and a wrong answer are different events and conflating them would obscure precisely the distinction this study exists to examine.

3.4 Integrator pinning

Every engine supplies only its right-hand side. Integration is performed by a single call to one explicit Runge–Kutta scheme (DOP853) with identical relative and absolute tolerances (rtol = 10^{-10}, atol = 10^{-12}) for all engines.

Pinning has a consequence that must be stated plainly: it makes the measurement structurally incapable of ranking the engines on integration accuracy. Energy drift under a pinned integrator is dominated by the integrator the engines share, not by the engines. This is not a limitation of the design but its purpose. Identical drift across engines is evidence that their equations are equivalent, which is what the comparison exists to establish. Without pinning, one engine's native integrator would have been compared against another's, and the difference attributed to the engines.

The design has a corresponding limitation, disclosed here rather than in the results. An engine whose constrained dynamics are enforced inside its own solver cannot participate in a pinned-integrator comparison on constrained systems without being asked to do something other than what it is built to do. Where this arises, the affected engine is scored on constraint satisfaction alone, and the asymmetry is reported rather than concealed. This is the case for the recursive library on the slider–crank (§5.6); the same engine participates fully on the cart–pole, which is a tree and requires no constraint.

3.5 Adjudication criteria

Two quantities are compared. The first is the residual between an engine's computed accelerations and the reference's, evaluated at 16 probe states per case drawn from the constraint manifold where one applies. The second is energy drift over a fixed integration horizon under the pinned integrator. For constrained cases a third quantity, constraint residual drift from $t = 0$, is recorded. The adjudication tolerance throughout is 10^{-8} relative. In the vocabulary of §2.3, the first quantity is the code-verification measurement proper — it compares derived equations against an exact standard — while the second and third are consistency checks on the integrated trajectory and are not independent of the shared integrator.

Constraint residual is measured as drift from the initial value rather than as an absolute quantity. The suite's constrained initial conditions are rounded to four decimal places, so every constrained case begins approximately 10^{-4} off its own constraint manifold. This is harmless for the classifier as constructed, but a reader checking the absolute residual would find an apparent violation and might reasonably attribute it to the engine. The fact is recorded here so that it cannot mislead.

3.6 Threats to validity

Common authorship of engine and referee. The referee shares no library with any engine, but it does share an author with one of them. A misconception held by that author could in principle appear in both, and their agreement would then be uninformative. Two considerations bound this risk. The

two engines the author did not write agree with the reference to the same tolerance, and a shared misconception would have to be present in all four independently.

Separately, the study record documents **four** instances in which the author's expectation about the *measuring apparatus* was contradicted by an independent artefact, and corrected before the affected results were reported:

Table 1: Faults in the measuring apparatus, each contradicted by an independent artefact and corrected before the affected results were reported.

#	Mistaken expectation	Caught by
1	The Hamiltonian pathway's right-hand side returns accelerations; it returns $\dot{p}$	discriminating $\dot{q}$ against $M^{-1}p$
2	A revolute chain reports absolute joint angles; it reports relative ones	inspecting the mass matrix
3	A pole offset rotated about $+y$ matches a reference using $x + l\sin\theta$; it mirrors it	inspecting the mass matrix
4	An engine reporting motion at an exact equilibrium is wrong; the exact solution is unreachable in finite precision	the independent reference behaving identically

Instances 1–3 are convention mismatches in the comparison and are the basis of the diagnostic in §9; instance 4 is a mistaken adjudication criterion. All four concern the instrument, and all four were detected by consulting an artefact rather than by reasoning further.

The study record contains further corrected errors that concern external facts rather than the instrument — chiefly the provenance of a third-party API guard, established only after documentation, a cached search result, and an unvalidated repository query had each produced a different wrong answer. Those are omitted here. They bear on the author's diligence, not on whether the measurement detects its own faults, and only the latter is evidence.

Mechanism coverage. Three mechanism families are examined. They were chosen for structurally distinct degeneracies — a vanishing transmission ratio, a collapsing mass-matrix determinant, and a chaotic regime reached by amplitude — but they remain three families, and the results do not license claims about mechanism classes outside them.

A comparator that is not independent on one family. On the cart–pole the computer-algebra package's residual against the reference is *exactly zero at all twenty configurations*, not merely small. This is addressed in §5.6: on that family the two are not independent, and the effective comparison is three-way rather than four-way.

Wall-clock timeouts. The reach results rest on a wall clock measured on a single machine in single runs, so the specific link counts at which an engine ceases to return are machine-dependent and should be read as such.

Driving style confounds capability comparison. This threat was realised rather than avoided, and is reported because it bears on how the reach results should be read. Each engine was initially exercised along a single path, and for the computer-algebra package that path was the one its documentation prescribes, which routes through a symbolic linear solve whose cost grows explosively. The resulting ladder was read as a capability separation. Driving each engine along the best path available through its own public interface removes the separation entirely (§6). A comparison of engines on capability

is therefore a comparison of engines-as-driven, and the choice of path belongs in the method rather than being left implicit. The correctness results are not affected: they adjudicate each engine against an independent reference rather than against each other.

Non-portable axes. One axis of the frozen matrix — symbolic nesting depth — is meaningful only to an engine that consumes raw Lagrangians, and no library-independent reference for it can exist. Those cases are excluded from the cross-engine claim by construction rather than by choice.

Constrained continuous-time dynamics in the recursive library. Constrained dynamics in the recursive library are enforced within its own discrete solver and are not available through the continuous-time interface used elsewhere in this study. The engine is therefore driven in its discrete mode on the slider–crank, where it cannot share the pinned integrator and is scored on constraint satisfaction alone. This is a property of the engine's supported interfaces and is reported as a limitation of the comparison; no defect claim is made here.

3.7 Disclosure

The author maintains one of the three engines measured. The mitigation is structural rather than declarative: no engine adjudicates itself or any other, the referee shares no library with any of them, the case matrix was frozen before measurement under a rule forbidding post hoc removal, and the results reported include the measurements on which the author's engine performs worst (§8.5).

AI assistance was used for portions of the implementation and analysis; see the disclosure statement.

All figures reported were produced by executing committed code. Artefacts — the case matrix, the reference implementation, the adapters, the sweep scripts, and the raw output records — are archived at doi:10.5281/zenodo.22315172 under the MIT licence.

4 Systems

Three mechanism families are examined. They were chosen so that each presents a degeneracy arising from a different mechanism, on the reasoning that a comparison exercising one kind of hard case repeatedly establishes less than one exercising several kinds once.

4.1 The planar N-link chain

An open chain of N links under gravity, with revolute joints and no constraint. The chain scales in degrees of freedom without changing its structure, which makes it the natural vehicle for the reach measurements of §6, and its dynamics pass from regular to chaotic as the initial amplitude increases, which makes it the vehicle for the amplitude axis.

The degeneracy here is dynamical rather than geometric. At the suite's original initial angles the motion is quasi-periodic: perturbing the initial state by 10^{-10} and integrating for ten seconds produces growth of order 10^{0}. At larger amplitudes the same perturbation grows by eleven orders of magnitude over the same horizon. Growth of order 10^{0} is polynomial and indicates regular motion; growth of order 10^{11} is exponential and indicates chaos.

The amplitude axis was added after measurement showed that the frozen suite had been exercising the chain only in its regular regime — that is, that a suite whose stated method is to increase difficulty until behaviour breaks down had been operating entirely within the well-behaved region of its principal system. The axis was declared before it was measured, in accordance with the governance rule, and the Lagrangian it uses is character-for-character the one already frozen; only the initial conditions differ.

4.2 The slider–crank

A crank rotating about a fixed pivot, connected by a rigid rod to a slider constrained to a straight line. The mechanism introduces two features absent from the chain: a closed kinematic loop, and a prismatic

joint alongside a revolute one.

The constraint admits a closed form. Expanding $|S - P|^2 - l^2$ yields $x^2 - 2rx\cos\theta + r^2 - l^2$, the trigonometric identity eliminating the remaining terms, so no symbolic algebra is required to construct the reference.

The degeneracy is geometric and arises from the transmission ratio. The effective inertia

$$M_{\text{eff}}(\theta) = m_c r^2 + m_s \left(\frac{\mathrm{d}x}{\mathrm{d}\theta}\right)^2 \tag{2}$$

varies by four orders of magnitude over a single revolution when the slider is heavy, because $\mathrm{d}x/\mathrm{d}\theta$ vanishes exactly at $\theta = 0$ and $\theta = \pi$. At those two configurations — the dead centres — the slider ceases to contribute inertia altogether and the mechanism approaches the loss of its mass matrix. The measured variation is 1.12×10^2 at $m_s/m_c = 10^2$, 1.11×10^4 at 10^4, and 1.11×10^6 at 10^6. The degeneracy is produced by the geometry rather than by a hand-constructed ill-conditioned matrix, which is what makes it a fair test.

Two parameters were swept: the rod-to-crank length ratio, from 3.0 down to 1.01 as the mechanism approaches its folding configuration, and the slider-to-crank mass ratio to 10^6. Engines were evaluated both at random states on the constraint manifold and exactly at both dead centres.

4.3 The cart–pole

A cart translating on a horizontal prismatic joint with a pole suspended from it by a revolute joint. Unlike the slider–crank the system is a tree, with no loop and no constraint, which permits all three engines to be compared in their ordinary modes of operation under a single pinned integrator, with nothing excluded for reasons unrelated to dynamics.

The system is structurally new to the study in two respects: it combines joint types within one mechanism, and its mass matrix is genuinely configuration dependent, where the suite's earlier multi-body systems had constant ones.

Its degeneracy arises from a third mechanism again. The determinant

$$\det M(q) = ml^2 \left(M + m\sin^2\theta\right) \tag{3}$$

collapses by six orders of magnitude twice per swing when the cart is light and the pole passes through vertical, because a near-massless cart cannot resist the pole's horizontal reaction. The slider–crank degenerates because a transmission ratio vanishes; the cart–pole degenerates because an inertia vanishes. The causes are unrelated, which is the reason for testing both.

5 Results: agreement

5.1 Equations of motion

Residuals between each engine's computed accelerations and the reference's are constant across the amplitude axis, as expected: the equations of motion do not depend on where the system starts.

Table 2: Equation-of-motion residuals against the reference, three-link chain, 16 probe states drawn uniformly from $[-0.5, 0.5]$. Relative to the double-precision unit round-off $u \approx 2.22 \times 10^{-16}$.

Engine	Residual	Multiple of u
MechanicsDSL	1.887×10^{-15}	8.5
Drake	1.072×10^{-14}	48
SymPy	2.620×10^{-14}	118

All three engines are therefore correct to the limit of the arithmetic, and the differences between them reflect accumulated rounding over differing operation counts rather than any difference in the equations derived.

5.2 Robustness of the probe sampling

Adjudication compares accelerations at probe states, so agreement could in principle be an artefact of where those states were drawn. The comparison was therefore repeated with five seeds and four sampling regimes chosen to stress different mechanisms: the study's own moderate box; velocities an order of magnitude larger, where Coriolis terms dominate and any error in the $C(q,\dot{q})\dot{q}$ bracket is amplified; angles clustered within 10^{-3} of π, where the gravity terms nearly cancel; and nearly aligned angles, where a serial chain's mass matrix is worst-conditioned and the linear solve most fragile.

Table 3: Worst relative disagreement with the reference over 5 seeds $\times$ 64 probe states per cell, each engine driven along its best available path. 19,200 comparisons in total.

N	Engine	uniform	wide	near-π	aligned
3	MechanicsDSL	1.0×10^{-14}	3.0×10^{-14}	2.9×10^{-16}	2.5×10^{-14}
	SymPy	8.0×10^{-15}	2.9×10^{-14}	1.3×10^{-16}	1.6×10^{-14}
	Drake	2.8×10^{-14}	4.1×10^{-14}	9.0×10^{-16}	8.1×10^{-14}
8	MechanicsDSL	1.9×10^{-14}	1.7×10^{-13}	1.7×10^{-16}	1.4×10^{-14}
	SymPy	3.4×10^{-14}	3.1×10^{-13}	1.7×10^{-15}	6.4×10^{-14}
	Drake	9.3×10^{-13}	6.1×10^{-13}	8.9×10^{-14}	1.6×10^{-12}
12	MechanicsDSL	5.2×10^{-14}	4.0×10^{-13}	3.3×10^{-16}	2.8×10^{-14}
	SymPy	9.4×10^{-14}	6.2×10^{-13}	7.6×10^{-15}	1.3×10^{-13}
	Drake	6.5×10^{-12}	2.8×10^{-12}	4.6×10^{-13}	5.0×10^{-12}

No cell exceeds the 10^{-8} adjudication tolerance, and the worst disagreement anywhere in 19,200 comparisons is 6.5×10^{-12}. Agreement is not an artefact of the sampling: it survives a tenfold increase in velocity range, the near-cancellation of gravity near π, and the worst-conditioned configurations a serial chain admits.

Two features of the table are worth noting rather than passing over. The near-π column shows the *smallest* residuals of the four regimes, because the accelerations there are themselves near zero and the comparison is relative; this is the regime in which §7 nonetheless finds every implementation failing, which is precisely the point that residual agreement cannot detect a shared failure. And the recursive library's residuals are consistently one to two orders larger than the symbolic engines', growing with N — expected of a recursive numerical algorithm against a closed form, and far inside tolerance at every point.

5.3 The amplitude axis

Across eight amplitudes spanning perturbation growth from single figures to 5.4×10^{11}, and three engines, 24 engine–amplitude pairs were evaluated. There were no refusals, no incorrect equations, and no integration failures.

Energy drift rises with amplitude, as harder dynamics demand more of the integrator, and does so almost identically for all three engines: at the worst case the three values are 4.74, 4.77 and 7.78×10^{-10}, the same quantity to within a factor of 1.6.

Table 4: Amplitude axis, three-link chain. Perturbation growth is the factor by which a 10^{-10} initial displacement grows over 10 s, measured on the reference at rtol = 10^{-12}. Energy drift is relative, under the pinned integrator.

Amp. (rad)	Growth	Regime	MechanicsDSL	SymPy	Drake
0.15	2.3	regular	7.302×10^{-11}	7.328×10^{-11}	7.445×10^{-11}
0.50	3.9	regular	2.201×10^{-11}	2.201×10^{-11}	2.201×10^{-11}
1.00	3.8	regular	2.702×10^{-11}	2.702×10^{-11}	5.123×10^{-11}
1.50	3.5×10^{4}	chaotic	1.723×10^{-10}	1.723×10^{-10}	2.860×10^{-10}
2.00	1.1×10^{7}	chaotic	2.137×10^{-10}	2.137×10^{-10}	2.139×10^{-10}
2.50	2.8×10^{10}	chaotic	4.737×10^{-10}	4.767×10^{-10}	7.783×10^{-10}
3.00	1.7×10^{11}	chaotic	3.036×10^{-10}	3.036×10^{-10}	3.042×10^{-10}
π	5.4×10^{11}	chaotic	2.737×10^{-10}	2.737×10^{-10}	2.737×10^{-10}

5.4 The chaotic regime

At three degrees of freedom with all links started well into the chaotic regime, energy drift is indistinguishable across the reference and all three engines.

Table 5: Chaotic regime, three-link chain, all links started at 3.0 rad. Perturbation growth 1.26×10^{11} over 10 s at rtol = 10^{-10}, implying a Lyapunov rate $\lambda = 2.556\,\mathrm{s}^{-1}$. Divergence is the time at which the trajectory departs the reference's by 10^{-6}; §8.3 explains why it is not a ranking.

Engine	Energy drift	Divergence
reference	3.0358×10^{-10}	—
MechanicsDSL	3.0356×10^{-10}	6.96 s
SymPy	3.0361×10^{-10}	6.28 s
Drake	3.0421×10^{-10}	5.69 s

The largest separation is 0.2 per cent relative, twelve orders of magnitude below any physically meaningful quantity.

5.5 The constrained pathway

MechanicsDSL agrees with the closed-form reference of equation (1) on all eight constrained cases in the adjudicated set. It is the only engine scored here: the other two expose no interface for imposing holonomic constraints on a coordinate set supplied directly, so this pathway is single-engine by construction. Their constrained behaviour is measured instead on the slider–crank (§5.6), where the constraint is intrinsic to the mechanism and each engine can impose it in its own idiom.

The redundancy family is the more informative half. Its constraint Jacobian retains rank one while its row count grows to nine, so the multipliers become underdetermined while the accelerations remain unique. The engine tracks the reference throughout. An exactly redundant constraint set does not determine how the constraint force is distributed, only its total, and neither the engine nor the reference is troubled by this.

The Hamiltonian pathway. The canonical route is the thinnest part of the study's coverage: the frozen ladder reached three coordinates before walling, and until now the pathway had no independent check at all, only energy conservation. It was therefore adjudicated directly in canonical coordinates, comparing the engine's $\dot{p}$ against the reference's own Legendre transform $p = M(q)\dot{q}$ at 64 probe states per size. The pathway agrees where it is reachable — 0 at one coordinate and 2.7×10^{-15} at two

Table 6: Constrained pathway, MechanicsDSL against the saddle-point reference, 16 probe states per case. `redund_R`k carries $k + 1$ constraints on two coordinates, all scalar multiples of one another.

Case	Coordinates	Constraints	Residual
`loops_N3`	4	3	5.788×10^{-16}
`redund_R0`	2	1	1.958×10^{-16}
`redund_R1`	2	2	1.084×10^{-15}
`redund_R2`	2	3	1.234×10^{-14}
`redund_R3`	2	4	2.554×10^{-15}
`redund_R4`	2	5	2.004×10^{-15}
`redund_R6`	2	7	3.266×10^{-15}
`redund_R8`	2	9	7.230×10^{-15}

— with the transform self-consistent to 1.1×10^{-16}, and walls at three coordinates as the frozen ladder records. No disagreement was found on the pathway most likely to harbour one.

The first attempt at this measurement reported a disagreement of 0.517 at two coordinates. It was instance 1 of Table 1 a fourth time: the engine's canonical right-hand side is a function of (q, p) and had been given $(q, \dot{q})$. The error vanished exactly at one coordinate, where $M = I$ makes the two coincide, which is the second tell described in §9 and the one that applies when the magnitude is not a clean 1 or 2. It is recorded here because it is the same fault recurring in the same place under a new harness, which is evidence about how easily the convention is confused rather than about any engine.

Coverage. With these cases adjudicated, the number of adjudicated cases carrying an independent derivation check rises from **22 to 40** of 45. The arithmetic does not compose as a simple increment, and the reconciliation is given in Table 7 because the figure is citable.

Table 7: Coverage reconciliation over the 45 adjudicated cases. The two referees cover overlapping but different sets, so the increment is not additive.

Referee	Cases	Composition
Computer-algebra oracle (original)	22	Lagrangian pathway only, including 5 symbolic-axis cases
Library-independent reference	35	`dof` 7, `mass_ratio` 14, `redundancy` 7, `near_singular` 6, `loops` 1
Union: any independent check	**40**	
gained by the reference	18	
reachable only by the original oracle	5	symbolic-axis Lagrangian cases
Carrying no independent check	**5**	symbolic-axis Hamiltonian cases

The five cases that remain unchecked are the Hamiltonian half of the symbolic axis. No library-independent reference for that axis can exist: nested cosine depth is meaningful only to an engine that consumes a raw Lagrangian, so there is no system to derive independently.

5.6 The slider–crank and the cart–pole

Across both additional families, 72 engine–case pairs were evaluated with no disagreements and no refusals.

Table 8: Cart–pole, five cart-to-pole mass ratios × four initial angles × three engines. Residuals are constant across the four angles at each mass ratio. Energy drift is the worst over the four angles.

		Residual			
M/m	det M range	MechanicsDSL	Drake	SymPy	Worst drift
10^{1}	1.1	3.72×10^{-16}	2.78×10^{-17}	0	1.99×10^{-11}
10^{0}	2.0	3.72×10^{-16}	1.86×10^{-16}	0	3.75×10^{-10}
10^{-2}	10^{2}	5.02×10^{-15}	2.05×10^{-16}	0	1.60×10^{-9}
10^{-4}	10^{4}	7.49×10^{-15}	5.55×10^{-17}	0	1.11×10^{-9}
10^{-6}	10^{6}	1.25×10^{-14}	1.96×10^{-16}	0	3.36×10^{-9}

All three engines agree with the reference at every mass ratio and every starting angle, including where the determinant collapses by six orders of magnitude, and energy drift is of order 10^{-9} or better throughout.

The computer-algebra package is not an independent comparator on this family. Its residual against the reference is *exactly* zero — not small — at all twenty configurations, over all sixteen probe states each. A residual of exactly zero under a relative-error metric means the two implementations produced bit-identical floating-point results, which indicates that they executed the same arithmetic in the same order rather than that they agree by independent derivation. The likely cause is that the cart–pole's mass matrix and forcing vector are simple enough that the package's symbolic simplification reproduces the hand-derived closed form exactly, and its code generation then emits the same sequence of operations.
Consequently the cart–pole comparison is effectively three-way — reference, symbolic compiler, recursive library — not four-way. This does not affect the other two families, where the same package returns non-zero residuals (2.62×10^{-14} on the chain, 5.55×10^{-16} to 1.96×10^{-15} on the slider–crank) and is a genuine independent comparator.

Subject to that qualification, the recursive library returns the smallest residuals on this family, of order 10^{-16} — better than the symbolic compiler by two orders of magnitude. This is the expected outcome when an algorithm designed for arbitrary topologies is applied to a two-body tree, and it is the converse of the pattern observed on the longer chains, where its residuals are the largest.

Both symbolic engines hold the rod length to approximately 10^{-9} through dead-centre crossings at a length ratio of 1.05 and a mass ratio of 10^{4}. The recursive library holds the rod to approximately 10^{-6} in its discrete mode.

5.7 Summary

No engine returned a wrong answer on any adjudicated case, in any family, in either dynamical regime, on either the constrained or the unconstrained pathway. Claim (B) of §1.3 — that engines differ in whether they report success on incorrect physics — is unsupported by every measurement in this study.

6 Results: divergence in reach

Agreement on cases all three engines can answer leaves open whether they can answer the same cases at all. Each engine was therefore taken up a ladder of system sizes to 30 links, with each engine–pathway–size combination built in its own subprocess under a fixed 120 s wall-clock limit, and the time measured to produce the equations of motion and evaluate the accelerations once. Results were checked against the reference wherever an answer was returned.

Read on its own, this ladder appears to show three distinct capability regimes: 5 links, 12 links, and 30, the largest size tested. §6.1 shows that reading to be wrong. Each engine here was driven

Table 9: Slider–crank. Residual and energy drift under the pinned integrator for the two engines that can share it; the recursive library is driven in its discrete mode and scored on constraint satisfaction alone (§3.4). Constraint column is the worst rod-length residual over a 5 s integration.

l/r	m_s/m_c	Engine	Residual	Energy drift	Constraint
3	10^2	MechanicsDSL	1.22×10^{-15}	1.51×10^{-10}	1.81×10^{-10}
3	10^2	SymPy	5.55×10^{-16}	1.51×10^{-10}	1.81×10^{-10}
3	10^2	Drake (disc.)	—	—	1.77×10^{-6}
1.5	10^2	MechanicsDSL	4.17×10^{-15}	2.11×10^{-10}	9.12×10^{-10}
1.5	10^2	SymPy	7.53×10^{-16}	2.12×10^{-10}	9.12×10^{-10}
1.5	10^2	Drake (disc.)	—	—	1.25×10^{-7}
1.05	10^2	MechanicsDSL	5.66×10^{-15}	5.40×10^{-10}	1.40×10^{-9}
1.05	10^2	SymPy	1.96×10^{-15}	5.40×10^{-10}	1.40×10^{-9}
1.05	10^2	Drake (disc.)	—	—	1.42×10^{-7}
3	10^4	MechanicsDSL	1.97×10^{-15}	2.92×10^{-10}	8.15×10^{-10}
3	10^4	SymPy	5.55×10^{-16}	2.90×10^{-10}	8.11×10^{-10}
3	10^4	Drake (disc.)	—	—	1.14×10^{-6}

along a single path, and for the computer-algebra package that path was the one its documentation prescribes, which routes through a symbolic linear solve whose cost grows explosively in the number of coordinates. The ladder measures those choices as much as it measures the engines.

Two observations from it survive that correction.

The recursive library is in a different class of behaviour. Its cost is flat at approximately 0.23 s from 5 links to 30, an $O(N)$ articulated-body algorithm set against two symbolic approaches whose cost grows explosively. Its residual grows with system size, from 6×10^{-15} at 3 links to 6×10^{-11} at 30, remaining four orders of magnitude inside the adjudication tolerance.

Cost is not monotonic in system size for the symbolic compiler: three links cost 21.63 s against 7.45 s for five. The effect is attributable to a threshold in the engine's symbolic-to-numeric handling and is reported rather than smoothed.

The finding that matters is what did not occur. Every engine that returned an answer returned a correct one, and every engine that could not, failed by never returning. A timeout is the loud failure mode: the researcher knows the computation did not finish. No engine returned a wrong answer at any size, and the study's original hypothesis remains unsupported on this axis as on every other.

6.1 The same systems, the same budget

The ladder above gave each engine one path. Giving each engine the best path available through its own public interface — Kane's formulation with a numeric solve and common-subexpression elimination for the computer-algebra package, simplification disabled for the symbolic compiler, and the single path the recursive library exposes — and asking all three for the same systems under the same wall clock in one session gives a different picture.

All three engines answer every rung to 80 coordinates, with no failures and no disagreements. The apparent capability separation of Table 10 does not survive equal treatment; 80 is where the ladder stopped, not where any engine failed.

What separates the engines is speed. The recursive library is flat at 0.2–0.3 s across the ladder and exceeds both symbolic engines by some 5500× at 80 coordinates — a difference of algorithm class, not of engineering quality, and one that no change of driving affects. Between the two symbolic engines the computer-algebra package driven well is about 1.6× faster at every rung, while the symbolic

Table 10: Scaling ladder, Lagrangian pathway, 120 s wall clock per engine–size pair. Single runs on one machine. Times include model construction and one acceleration evaluation. Once an engine fails to return it is not retried at larger sizes.

	MechanicsDSL		**SymPy**		**Drake**	
N	time	residual	time	residual	time	residual
1	1.57 s	0	0.33 s	0	0.44 s	0
2	2.17 s	1×10^{-16}	0.38 s	4×10^{-16}	0.24 s	2×10^{-16}
3	21.63 s	4×10^{-16}	0.59 s	8×10^{-15}	0.25 s	6×10^{-15}
5	7.45 s	2×10^{-16}	2.58 s	1×10^{-15}	0.24 s	7×10^{-14}
8	42.03 s	2×10^{-15}	*timeout*		0.22 s	5×10^{-14}
12	64.43 s	6×10^{-15}	—		0.22 s	1×10^{-12}
20	*timeout*		—		0.23 s	1×10^{-11}
30	—		—		0.23 s	6×10^{-11}

Table 11: Equal-budget reach. Each engine driven along the best path available through its public interface, same systems, same 5400 s wall clock, one session on one machine, adjudicated against the same reference at 6 probe states. Residual is the worst relative disagreement with the reference.

	MechanicsDSL		**SymPy**		**Drake**	
N	time	residual	time	residual	time	residual
40	145.0 s	1.7×10^{-13}	74.3 s	3.6×10^{-13}	0.3 s	1.7×10^{-10}
60	468.0 s	4.2×10^{-13}	280.2 s	7.1×10^{-13}	0.2 s	4.1×10^{-10}
80	1100.9 s	5.9×10^{-13}	693.3 s	1.2×10^{-12}	0.2 s	9.9×10^{-10}

compiler returns residuals about twice as small on this family. The residual advantage is family-specific: it reverses on the cart–pole and the slider–crank, and is reported here only to be qualified. Neither difference is a capability boundary, and every value involved is orders of magnitude inside the adjudication tolerance.

The generalisable point is not about these three engines. It is that a comparison of engines on capability is a comparison of engines-as-driven, and that the path taken through each tool belongs in the method rather than being left implicit. The confound here survived a pre-registered case matrix and a frozen baseline, neither of which was designed to catch it.

7 A failure shared by every implementation, including the referee

7.1 The configuration

Consider the N-link chain with every joint angle set to π and every velocity set to zero: the chain stands inverted, each link balanced directly above the one below. The exact solution is that nothing moves. Every term in the equations of motion vanishes analytically. The gravity term is proportional to $\sin \pi$, which is zero. The Coriolis term vanishes because the velocities are zero and the angles equal. The configuration is an equilibrium, and the exact trajectory is the constant function.

It is, of course, an *unstable* equilibrium. That distinction is the whole of what follows.

What is and is not claimed here. An objection presents itself immediately and should be answered before the result rather than after it. A physical pendulum released at the inverted position falls, so returning a large trajectory might be called right rather than wrong. The objection misses which system

is under test. The engines were not asked what a physical pendulum does; they were asked to integrate a stated initial-value problem, and for that problem — exact angles of π, exact zero velocities — the exact solution is the constant function. Every engine returns something else.

The claim is therefore not that the engines are defective, nor that the returned trajectory is unphysical. It is narrower and, for a user, worse: **every implementation returns an answer dominated by its own round-off, reports success, and gives no indication that it has done so.** A researcher who did not already know the initial condition was an unstable equilibrium would have no way of learning it from the output. That is the silent failure the study set out to look for, and it is the one case in which it was found.

7.2 The observation

All three engines, and the independent reference, return approximately 20 radians of motion over a ten-second horizon. All four report success. None emits a warning, a diagnostic, or any indication that the returned trajectory bears no relation to the exact solution of the system as specified.

Energy is conserved throughout to 2.737×10^{-10} — the same tolerance as in every other case in the study, and identical across all three engines (Table 4, final row). The trajectory is smooth, physically plausible in isolation, and entirely wrong. **That energy is conserved so well during the fall is precisely why every check in the suite passes it.**

7.3 The mechanism

The cause is arithmetic rather than algorithmic. In binary64, $\sin(\pi)$ evaluates not to zero but to $1.2246{\times}10^{-16}$, because π is not exactly representable. This leaves a residual gravitational acceleration of 1.20×10^{-15} at the initial state. The equilibrium being unstable, that residual is amplified exponentially, and within the ten-second horizon it reaches full scale.

This raises an alternative the argument so far does not exclude. Every engine and the reference ran at the same precision on the same hardware, so the failure might be shared because the substrate is shared rather than because the phenomenon is structural. The two predict different things and the difference is measurable: if the failure is an artefact of binary64, sufficient precision removes it; if it is structural, precision only delays it.

Integrating the perturbation in arbitrary precision settles it (Table 12). Linearising about the equilibrium gives $\ddot{\varphi} = \lambda^2 \varphi$ with $\lambda = \sqrt{g/l} = 3.1321\,\mathrm{s}^{-1}$, so a residual $\varepsilon \sim 10^{-p}$ at p working digits should reach a fixed threshold after a time linear in p, with slope $\ln 10/\lambda = 0.735\,\mathrm{s}$ per digit. That is what is observed, to the integration step, across nine orders of magnitude in precision.

Table 12: Divergence from the inverted equilibrium against working precision. Fixed-step RK4 in arbitrary precision, perturbation coordinates, threshold 0.1 rad. Predicted time is $\operatorname{arccosh}(0.1/\varepsilon)/\lambda$.

Digits	$\varepsilon = \lvert \sin \pi_p \rvert$	**Measured**	**Predicted**
16	1.14×10^{-17}	11.94 s	11.94 s
25	1.88×10^{-26}	18.40 s	18.40 s
35	5.51×10^{-37}	26.14 s	26.14 s
50	1.01×10^{-51}	36.98 s	36.98 s
75	8.01×10^{-77}	55.43 s	55.43 s
100	2.87×10^{-102}	74.14 s	74.14 s
150	1.32×10^{-151}	110.41 s	110.41 s

The measured slope is 0.735 s per digit against a predicted 0.735, the relation is monotone in precision, and **at no precision tested does the failure disappear**. Precision buys delay proportional to the

number of digits and never buys immunity. **No finite-precision computation can remain at an unstable equilibrium**, and the behaviour is a property of the problem rather than a defect in any engine. That the independent reference — which shares no code with any engine and was derived by hand — exhibits the identical behaviour is what establishes this. Had the reference remained at rest, the finding would have been a defect in three engines. It did not, and the finding is something else.

7.4 Why the case is retained

An earlier version of this axis was written on the expectation that an engine reporting motion at the inverted equilibrium is wrong and an engine reporting no motion is right. That expectation was false. It is recorded here, and corrected in the module rather than silently revised, because the ease with which it was held is part of the finding: the intuition that a correct implementation should return the exact answer at an exact equilibrium is natural, widely shared, and incorrect. It is instance 4 of Table 1.

7.5 The same failure in a second mechanism

One instance of a shared failure is a curiosity; the claim that differential testing is blind to a *class* of failure requires more. The cart–pole supplies a structurally different test: a tree rather than a chain, mixed joint types, and an inverted equilibrium in which the cart is free to recoil, so the instability couples two coordinates rather than one.

Started with the pole vertical and everything at rest, at cart-to-pole mass ratios of 10, 1 and 10^{-2}, **all four participants — the three engines and the reference — report success and return motion**, and do so in agreement with one another to every digit displayed: 2.803×10^{-3} rad at $M/m = 10$, 6.002 rad at $M/m = 1$, 6.283 rad at $M/m = 10^{-2}$. Not one of the twelve cases stayed at rest, and not one warned.

The phenomenon is therefore not a property of the chain. Four independent implementations agree closely on a trajectory that is entirely round-off, in two mechanisms whose only common feature is that the initial condition is an unstable equilibrium.

What the case exposes is real and shared. Every implementation tested returns success and hands back a large trajectory where the exact answer is no motion at all, and none warns that the initial condition is an unstable equilibrium whose evolution is dominated by round-off. This is the study's original thesis appearing as a universal blind spot rather than as a difference between engines. It is a weaker result than a disagreement would have been, and an honest one.

7.6 What follows for differential testing

Differential testing rests on the premise that independent implementations fail independently, so that disagreement localises incorrectness. This case is a counterexample of a specific kind. The failure is not independent across implementations because its cause is not in any implementation: it is in the interaction between the problem and the arithmetic every implementation shares.

Cross-implementation comparison cannot detect a failure of this kind, because every implementation exhibits it identically. Adding a fourth engine, a fifth, or a reference derived from first principles by an independent author would not help — this study added exactly such a reference, and it failed in the same way.

The class of failures invisible to differential testing is therefore not empty, and it is not exotic. Unstable equilibria are not pathological configurations contrived for a test suite; they are the operating point of any inverted-pendulum control problem and appear throughout robotics and biomechanics. A method that cannot see them has a blind spot in a region practitioners actually work in.

8 What these measurements do and do not support

8.1 The integrator was pinned, which removes the variable

Every engine supplied only its right-hand side, and integration was a single call with identical tolerances for all of them. Energy drift is therefore dominated by the shared integrator, not by the engines. Identical drift is evidence that the equations are equivalent, which is what pinning was intended to establish. The measurement is structurally incapable of ranking the engines on integration accuracy, and that was its purpose.

8.2 The differences are not differences

The worst energy-drift values across the amplitude axis are 4.74, 4.77 and 7.78×10^{-10} — the same quantity to within a factor of 1.6. In the chaotic case the three values differ by 0.2 per cent relative, twelve orders of magnitude below any physically meaningful quantity. No ranking can be built on separations of this size.

8.3 Divergence time cannot rank the engines

In a chaotic system the time at which one trajectory departs from another is governed by the size of the initial difference between their right-hand sides, amplified exponentially. With the Lyapunov rate $\lambda = 2.556\,\mathrm{s}^{-1}$ implied by the measured growth, a seed s should cross a 10^{-6} threshold at $t = \lambda^{-1} \ln(10^{-6}/s)$.

Engine	N=3 **residual**	**Predicted**	**Observed**
MechanicsDSL	1.89×10^{-15}	7.86 s	6.96 s
Drake	1.07×10^{-14}	7.18 s	5.69 s
SymPy	2.62×10^{-14}	6.83 s	6.28 s

The band is predicted; the ordering is not. Predicted times span 6.8–7.9 s and observed times span 5.7–7.0 s, so the mechanism is right — but the predicted order is MechanicsDSL, Drake, SymPy while the observed order is MechanicsDSL, SymPy, Drake. Drake and SymPy transpose.

The reason the ordering fails is instructive, and it strengthens rather than weakens the conclusion. The three seeds span barely one order of magnitude while the amplification spans eleven. The effective seed is also the residual *along the actual trajectory*, not the maximum over random probe states, and the threshold crossing is itself noisy in a chaotic system. Divergence time therefore has no resolving power between engines whose equations agree this closely: it measures the Lyapunov exponent, with the engine identity as a rounding error.

8.4 Why one engine's residual is larger, and why that is not worse

The recursive library evaluates dynamics with an articulated-body algorithm built for arbitrary topologies, contact, and real-time control. The reference and the symbolic engines perform a direct solve of one hand-derived Lagrangian for one fixed topology. More arithmetic operations accumulate more rounding. At 7×10^{-14} on the longer chains this is a deliberate generality-for-precision trade, not a defect, and it remains six orders of magnitude inside the adjudication tolerance. The cart–pole results of Table 8, where the same engine returns the smallest residuals of the three, confirm the interpretation.

8.5 Where the engines genuinely differ, the author's engine is behind

Compile time at three links is approximately 37 times slower than the computer-algebra package (21.63 s against 0.59 s, Table 10). The Hamiltonian pathway times out from three links, where the other engines

do not. On the near-singular axis its accuracy is 5.0×10^{-12} against 1.7×10^{-15}. On the cart–pole its residuals are two orders of magnitude larger than the recursive library's. These measurements are reported for the same reason as the rest: a study that reported only the comparisons its author's engine wins would not be worth reading.

8.6 What can be claimed

That the three engines agree with an independent closed-form derivation, and with one another, to within $\mathbf{3 \times 10^{-14}}$ on equations of motion, and conserve energy indistinguishably under a pinned integrator, across amplitudes spanning the regular and chaotic regimes, across three mechanism families, and across constrained and unconstrained pathways.

The bound is set by the largest residual anywhere in the study — the computer-algebra package at 2.62×10^{-14} on the three-link chain (Table 2) — not by the smallest. It is stated as 3×10^{-14} rather than more tightly because a bound that does not cover every measurement is not a bound.

That is an independently adjudicated correctness result. A claim of superiority for any one engine would rest on differences of a fraction of a per cent in a metric governed by a shared integrator, and — in the case of the author's engine — would be advanced by the party with an interest in it. Such a claim would be identified immediately and would discredit the work that is sound.

9 A diagnostic for convention mismatch

Three times during this study a comparison produced an error whose pattern across a sweep, rather than its magnitude, revealed the cause. On each occasion that cause was a convention mismatch between the reference and the engine adapter, not a defect in the engine. Two of the three produced an error identical across every case to many significant figures; the third did not, and the difference between them turns out to matter.

Observed error	**Cause**	**Degenerate case that hid it**
4.53, and exactly 1.00 at dead centre	Hamiltonian pathway integrates (q, p) and returns $\dot{p}$; compared against $\ddot{q}$	N=1, where $M = I$ makes $p = \dot{q}$
2.5–6.4, growing in N	engine reports *relative* joint angles; reference uses *absolute*	N=1, where the two coincide
exactly 2.00, all 20 cases	pole offset rotated about $+y$ mirrors the mechanism, flipping the cart's acceleration	—

A discrepancy arising from genuine numerical difference varies with configuration, because the quantities producing it vary with configuration. A discrepancy that is constant across cases with different parameters cannot have that origin: it indicates that two implementations are being asked different questions, and that the comparison rather than the implementation is at fault.

How far the rule actually reaches. Because that rule was inferred from the three incidents that produced it, it was tested against faults of known type: four convention faults and three arithmetic ones injected into the reference and scored by the study's own metric over 15 systems of differing size and mass. The result qualifies the rule in one direction and confirms it in the other.

No arithmetic fault produced a clean constant (0 of 3), so the rule does not raise false alarms. But only two of four convention faults did. Permuted coordinates and relative-versus-absolute angles both vary with configuration exactly as an arithmetic fault would — and the second of those is the third

Table 13: Injected faults of known type, scored by relative error over 15 systems. Spread is the coefficient of variation across systems.

Injected fault	Kind	Mean error	Spread	Clean 1 or 2?
sign flip	convention	2.0000	0	yes
term absent	convention	1.0000	0	yes
coordinates permuted	convention	73.15	1.06	no
relative vs. absolute	convention	14.91	0.82	no
mass entry $+10^{-6}$	arithmetic	10^{-5}	0.87	no
Coriolis dropped	arithmetic	0.634	0.98	no
gravity 1% wrong	arithmetic	0.0133	0.30	no

incident in the table above, which is why it was caught by inspecting the mass matrix rather than by the signature.

The rule is therefore sufficient but not necessary. A clean constant error is strong evidence of a convention mismatch; a varying error is no evidence against one. Stated the other way round it would have dismissed half the convention faults in this study as genuine numerical difference.

Under a relative-error metric the two commonest degenerate cases produce characteristic values. A term absent entirely from one side yields a relative error of exactly 1; a term present with the wrong sign yields exactly 2, since $|a_d - a_r| = 2|a_r|$ divides out to exactly 2 wherever $|a_r| > 1$. Both were observed here.

A second tell is that the error vanishes in a degenerate configuration and grows away from it. Genuine derivation errors rarely switch on at a particular system size; convention mismatches characteristically do, because degenerate cases make differing conventions coincide.

The rule is stated for use rather than for interest: *in a cross-implementation comparison, a discrepancy identical to many significant figures across cases with different parameters indicates a convention mismatch in the comparison, not a defect in the implementation under test.* Its practical value is that it fires early, before a spurious defect is reported. Each of the three instances here was resolved by inspecting the mass matrix directly, and each would otherwise have been reported as a defect in widely used software.

10 Conclusion

Three independently developed rigid-body dynamics engines were compared under adjudication by a closed-form reference sharing no library with any of them, across a frozen adversarial case matrix, three structurally distinct mechanism families, both dynamical regimes, and constrained and unconstrained pathways.

They agree to the limit of double-precision arithmetic. No engine returned an incorrect answer while reporting success on any adjudicated case, and every engine that failed did so by not returning rather than by returning something wrong.

An apparent difference in reach — the largest system each engine would answer — proved on examination to measure how each engine had been driven rather than what it could do. Given the best path through each engine's own public interface and an equal budget, all three answer systems of 80 coordinates (§6.1). That correction is reported rather than quietly applied, because a study of measurements that mislead has no standing to conceal one of its own.

The single failure observed is shared by all three engines and by the independent reference. At an unstable equilibrium every implementation reports success while returning a trajectory the exact solution does not have, and none warns. Its cause lies in the interaction between the problem and finite-precision arithmetic, not in any implementation.

The consequence for method is the paper's main contribution. Differential testing between implementations is a strong instrument for the failures implementations make independently, and this study found none of those in three mature engines. It is structurally blind to the failures they share. Where the shared failure arises from arithmetic rather than from code, adding implementations does not help, because the reference exhibits the failure too. Detecting such cases requires reasoning about the problem — here, recognising that the initial condition is an unstable equilibrium — rather than comparing implementations of it.

That the engines agree is a reassuring result for practitioners who rely on them. That they agree even when all of them are wrong is the more useful one.

Disclosure

The author maintains one of the three engines measured. This is disclosed in every document of the study. The mitigation is structural: no engine adjudicates itself or any other, and the referee shares no library with any of them. The case matrix was frozen before measurement under a rule forbidding post hoc removal of cases, and §8.5 reports the measurements on which the author's engine performs worst.

Use of AI assistance. An AI assistant (Anthropic Claude) was used for portions of the implementation, the analysis, and the drafting of parts of this manuscript. The study design, the adjudication criteria, the freeze protocol, the interpretation of the results, and every decision about what is and is not claimed are the author's, as is responsibility for the whole. Every reported figure was produced by executing committed code and checked against the run record.

Post-freeze changes to the author's engine. Diagnosing the reach confound located two defects in the author's engine: a dominant cost in a cosmetic simplification step, and a configuration control that reported bounding that cost while having no effect. Both were corrected after measurement. **The corrections are not present in the version measured here**, which remains pinned for the study; no figure in this paper reflects them. They are reported separately so that the improvement to the author's own tool is not presented as a result of a study in which that tool is a subject.

Artefacts. Every figure reported was produced by executing committed code. Artefacts — the case matrix, the reference implementations, the engine adapters, the sweep scripts, and the raw output records — are available at `doi:10.5281/zenodo.22315172`, archived under the MIT licence. The repository is at `https://github.com/MechanicsDSL/mechanicsdsl` (study material under `stress_suite/`).

References

[1] American Society of Mechanical Engineers. *ASME V&V 10-2019: Standard for Verification and Validation in Computational Solid Mechanics*. ASME, New York, 2019.

[2] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi:10.1109/TSE.2014.2372785

[3] T. Erez, Y. Tassa, and E. Todorov. Simulation tools for model-based robotics: Comparison of Bullet, Havok, MuJoCo, ODE and PhysX. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 4397–4404, 2015.

[4] F. González, D. Dopico, R. Pastorino, et al. Behaviour of augmented Lagrangian and Hamiltonian methods for multibody dynamics in the proximity of singular configurations. *Nonlinear Dynamics*, 85:1491–1508, 2016. doi:10.1007/s11071-016-2774-5

[5] L. Hatton and A. Roberts. How accurate is scientific software? *IEEE Transactions on Software Engineering*, 20(10):785–797, 1994.

[6] IFToMM Technical Committee for Multibody Dynamics. Library of computational benchmark problems. https://www.iftomm-multibody.org/benchmark. Accessed September 2026.

[7] U. Kanewala and J. M. Bieman. Testing scientific software: A systematic literature review. *Information and Software Technology*, 56(10):1219–1232, 2014. doi:10.1016/j.infsof.2014.05.006

[8] J. C. Knight and N. G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109, 1986.

[9] W. M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[10] W. L. Oberkampf and T. G. Trucano. Verification and validation in computational fluid dynamics. *Progress in Aerospace Sciences*, 38(3):209–272, 2002. doi:10.1016/S0376-0421(02)00005-2

[11] W. L. Oberkampf and C. J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, Cambridge, 2010. ISBN 978-0-521-11360-1.

[12] P. J. Roache. *Verification and Validation in Computational Science and Engineering*. Hermosa Publishers, Albuquerque, NM, 1998. ISBN 978-0-913478-08-0.

[13] P. J. Roache. Code verification by the method of manufactured solutions. *Journal of Fluids Engineering*, 124(1):4–10, 2002. doi:10.1115/1.1436090